UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA Nº 04

NOMBRE COMPLETO: Hernández Domínguez Luis Carlos

Nº de Cuenta: 320182668

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 21/09/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejercicios

Ejercicio 1: Terminar la grúa con:

-cuerpo (prisma rectangular

-base (pirámide triangular

-4 llantas (4 cilindros) con teclado se pueden girar las 4 llantas por separado

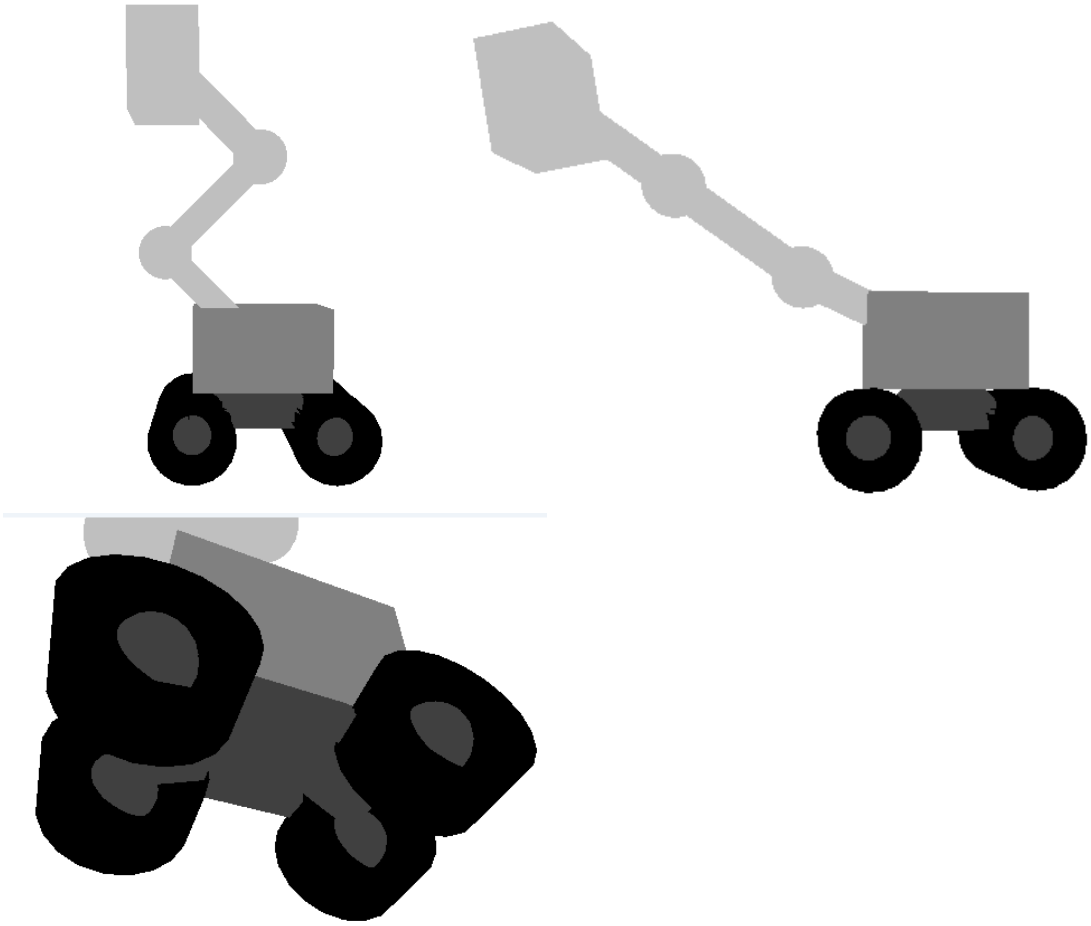
Bloques de código generado

```
463             model = modelaux = modelaux2;
464
465             //Base
466             model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
467             modelaux = model;
468             model = glm::scale(model, glm::vec3(5.0f, 2.5f, 3.0f));
469             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
470             color = glm::vec3(0.25f, 0.25f, 0.25f);
471             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
472             meshList[4]->RenderMeshGeometry(); //dibuja cubo y pirámide triangular
473
474             //articulación 5
475             model = modelaux;
476             model = glm::translate(model, glm::vec3(-2.5f, -1.5f, -1.5f));
477             model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
478             model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
479             color = glm::vec3(0.25f, 0.25f, 0.25f);
480             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
481             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
482             sp.render();
483
484             //Rueda 1
485             model = glm::scale(model, glm::vec3(2.2f, 2.2f, 2.2f));
486             model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
487             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
488             color = glm::vec3(0.0f, 0.0f, 0.0f);
489             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
490             meshList[2]->RenderMeshGeometry(); //dibuja cubo y pirámide triangular
```

```
492             //articulación 6
493             model = modelaux;
494             model = glm::translate(model, glm::vec3(-2.5f, -1.5f, 1.5f));
495             model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
496             model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, 1.0f));
497             color = glm::vec3(0.25f, 0.25f, 0.25f);
498             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
499             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
500             sp.render();
501
502             //Rueda 2
503             model = glm::scale(model, glm::vec3(2.2f, 2.2f, 2.2f));
504             model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
505             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
506             color = glm::vec3(0.0f, 0.0f, 0.0f);
507             glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
508             meshList[2]->RenderMeshGeometry(); //dibuja cubo y pirámide triangular
509
510             //articulación 7
511             model = modelaux;
512             model = glm::translate(model, glm::vec3(2.5f, -1.5f, -1.5f));
513             model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
514             model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
515             color = glm::vec3(0.25f, 0.25f, 0.25f);
516             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
517             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
518             sp.render();
```

```
520             //Rueda 3
521             model = glm::scale(model, glm::vec3(2.2f, 2.2f, 2.2f));
522             model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
523             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
524             color = glm::vec3(0.0f, 0.0f, 0.0f);
525             glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
526             meshList[2]->RenderMeshGeometry(); //dibuja cubo y pirámide triangular
527
528             //articulación 8
529             model = modelaux;
530             model = glm::translate(model, glm::vec3(2.5f, -1.5f, 1.5f));
531             model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
532             model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
533             color = glm::vec3(0.25f, 0.25f, 0.25f);
534             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
535             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
536             sp.render();
537
538             //Rueda 4
539             model = glm::scale(model, glm::vec3(2.2f, 2.2f, 2.2f));
540             model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
541             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
542             color = glm::vec3(0.0f, 0.0f, 0.0f);
543             glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
544             meshList[2]->RenderMeshGeometry(); //dibuja cubo y pirámide triangular
```

Ejecución



Para resolver este problema continuando a partir del ejercicio de clase fue necesaria una segunda variable auxiliar para poder regresar al cuerpo de la grúa y apartir de allí ir jerarquizando la base y las cuatro ruedas con sus respectivas articulaciones. De igual forma, fue necesario agregar más articulaciones en window.h y window.cpp.

Ejercicio 2: Crear un animal robot 3d

-instanciando cubos, pirámides, cilindros, conos, esferas:

-4 patas articuladas en 2 partes (con teclado se puede mover las dos articulaciones de cada pata)

-cola articulada o 2 orejas articuladas. (con teclado se puede mover la cola o cada oreja independiente)

Bloques de código generado

```
//Cuerpo
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 1.0f, 0.0f));
modelaux2 = modelaux = model;
model = glm::scale(model, glm::vec3(3.6f, 3.0f, 2.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.75f, 0.75f, 0.75f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

//Pata delantera derecha
//articulacion 1
model = modelaux;
model = glm::translate(model, glm::vec3(0.3f, 0.5f, 1.4f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.75f, 0.75f, 0.75f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

model = glm::translate(model, glm::vec3(-0.1f, -0.45f, 0.0f));
model = glm::rotate(model, glm::radians(350.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.8f, 2.6f, 1.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.25f, 0.25f, 0.25f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
```

```
//articulación 2
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.2f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.75f, 0.75f, 0.75f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

model = glm::translate(model, glm::vec3(0.7f, -1.35f, 0.0f));
model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.1f, 2.4f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.25f, 0.25f, 0.25f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

model = modelaux;
model = glm::rotate(model, glm::radians(340.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.5f, -1.08f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(2.0f, 0.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.25f, 0.25f, 0.25f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
```

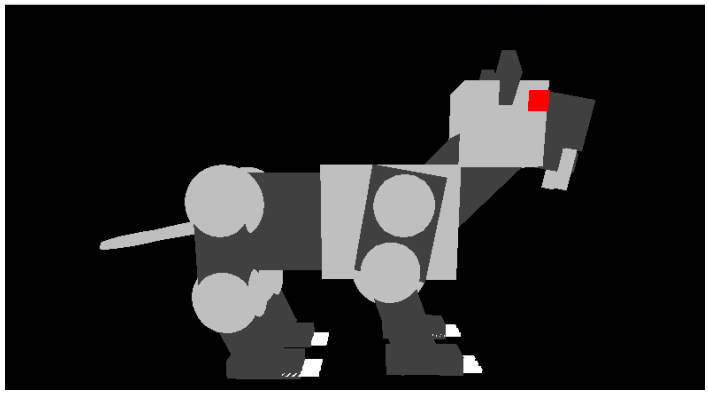
```
//garras
model = glm::translate(model, glm::vec3(0.5f, -0.25f, -0.4f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 0.2f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.4f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 0.2f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.4f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 0.2f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
```

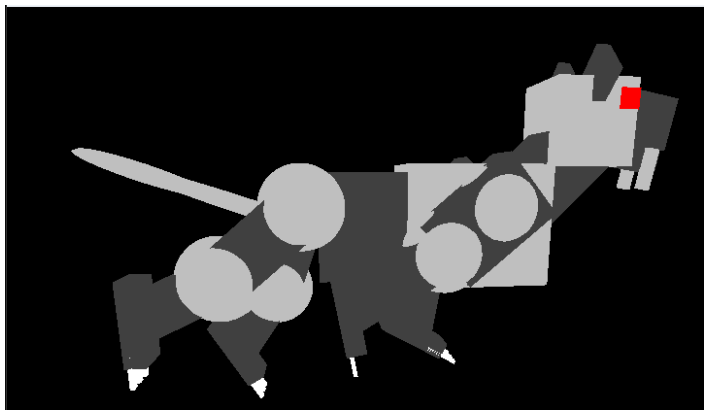
```
//segunda seccion del torso
model = modelaux;
model = glm::translate(model, glm::vec3(-2.7f, 0.0f, 0.0f));
modelaux2 = modelaux = model;
model = glm::scale(model, glm::vec3(2.5f, 2.6f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.25f, 0.25f, 0.25f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

//Pata trasera derecha
//articulacion 5
model = modelaux;
model = glm::translate(model, glm::vec3(-1.6f, 0.5f, 1.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.75f, 0.75f, 0.75f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.3f, 2.5f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.25f, 0.25f, 0.25f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
```

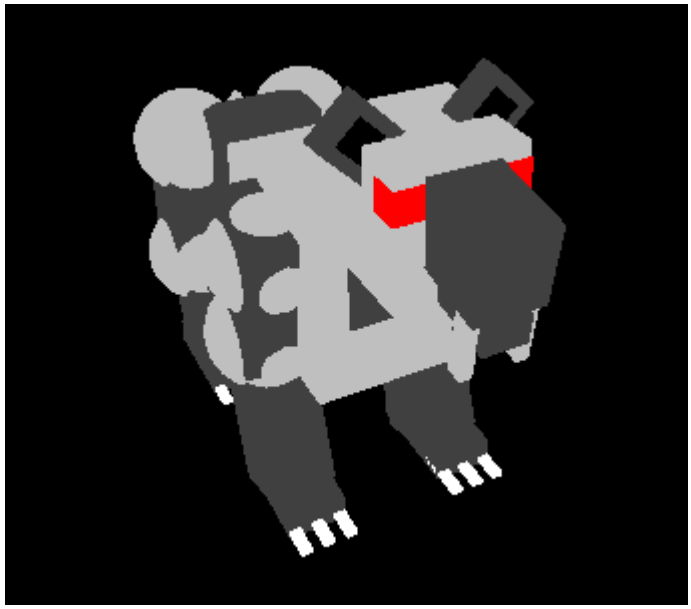
```
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.75f, 0.75f, 0.75f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

model = glm::rotate(model, glm::radians(25.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.1f, 2.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.25f, 0.25f, 0.25f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

model = modelaux;
model = glm::rotate(model, glm::radians(335.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.5f, -1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(2.0f, 0.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.25f, 0.25f, 0.25f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
```

```
//articulación 9
model = modelaux  = modelaux2;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();


model = glm::translate(model, glm::vec3(-2.75f, 0.0f, 0.4f));
model = glm::rotate(model, glm::radians(20.0f), glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(3.0f, 0.2f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.75f, 0.75f, 0.75f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();


model = modelaux;
model = glm::translate(model, glm::vec3(-2.65f, 0.0f, 0.0f));
//model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 0.2f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.75f, 0.75f, 0.75f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

Ejecución







Para resolver este problema de la misma manera que con la grúa fue necesario definir el nodo padre de toda la estructura que en este caso fue el cuerpo del animal robot en cuestión. E ir jerarquizando y añadiendo las articulaciones necesarias para el modelo.

2. Problemas presentados.

No se presentó ningún problema a la hora de ejecutar código.

3.- Conclusión:

a. Está práctica fue bastante interesante porque el modelo jerárquico facilita en algunos aspectos el modelado realizado en la práctica previa con el pyraminx. De igual forma, el intentar crear un modelo con solo usar cuerpos geométricos y jerarquizarlos para mover las extremidades correctamente fue algo bastante interesante y nuevo.
b. Los conceptos y explicaciones dadas en clase fueron correctas y suficientes para la resolución de esta práctica.
c. En conclusión, esta práctica me pareció tener un nivel de dificultad adecuado y hace un buen trabajo al introducir el modelo jerarquico.